

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ
И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Программная инженерия»

ОТЧЁТ

по Практике (Системы искусственного интеллекта)

Выполнил:

Студент группы: УБВТ2401

ФИО: Кошелев Ю. Д.



Проверил:

Старший преподаватель кафедры
«Программная инженерия»

А.Д. Липатова

Москва, 2026

ОГЛАВЛЕНИЕ

<u>ВВЕДЕНИЕ</u>	3
<u>РАЗДЕЛ 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ ПРОЕКТА</u>	6
<u>1.1. Объект исследования</u>	6
<u>1.2. Предмет исследования</u>	7
<u>1.3. Цель исследования</u>	8
<u>РАЗДЕЛ 2. ПРАКТИЧЕСКАЯ РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ</u>	11
<u>2.1. Постановка задач практической разработки</u>	11
<u>2.2. Используемые технологии и архитектура проекта</u>	12
<u>2.3. Описание реализации проекта Velo-O-Wisp</u>	14
<u>2.4. Организация базы данных</u>	16
<u>РАЗДЕЛ 3. РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ</u>	19
<u>3.1. Описание результата практической деятельности</u>	19
<u>3.2. Тестирование и анализ результатов</u>	22
<u>3.3. Ограничения проекта и возможные улучшения</u>	24
<u>ЗАКЛЮЧЕНИЕ</u>	27
<u>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ</u>	28
<u>ПРИЛОЖЕНИЯ</u>	29
<u>Приложение А. Репозиторий проекта</u>	29
<u>Приложение Б. Макет интерфейса проекта</u>	29

ВВЕДЕНИЕ

В ходе производственной практики по дисциплине «Практика (Системы искусственного интеллекта)» мной был выполнен учебный проект Velo-O-Wisp по теме «Учет велосипедов на парковке у метро». Работа была связана с применением методов компьютерного зрения для автоматического анализа изображения и подсчета объектов определенного класса.

Практическое задание заключалось в создании веб-приложения, которое принимает изображение от пользователя, передает его на серверную обработку, использует предобученную нейронную сеть для обнаружения велосипедов, отображает результат с ограничивающими рамками и сохраняет сведения о выполненном анализе. В качестве основного инструмента детекции была использована модель YOLOv8, так как она подходит для поиска объектов на изображениях и позволяет определить расположение найденных объектов в кадре.

Разрабатываемый проект имеет прикладной характер. Велосипедные парковки у станций метро используются как часть городской транспортной инфраструктуры. Для оценки их загруженности может потребоваться определение количества размещенных велосипедов. Ручной подсчет по изображению возможен, но при регулярном выполнении такой операции он занимает время и зависит от внимательности человека. Поэтому автоматизированная обработка изображения с использованием нейронной сети является подходящим способом решения данной учебной задачи.

Целью работы являлась разработка веб-приложения Velo-O-Wisp для автоматизированного обнаружения и подсчета велосипедов на изображении парковки у метро.

Для достижения цели были выполнены следующие задачи:

1. Проанализировано индивидуальное задание и определено, что проект должен решать задачу детекции объектов на изображении.
2. Выбрана предобученная модель YOLOv8 для обнаружения объектов класса bicycle.

3. Разработан пользовательский интерфейс для загрузки изображения с компьютера.
4. Реализована серверная часть приложения на Python с использованием Flask.
5. Настроена обработка изображений с использованием YOLOv8 и OpenCV.
6. Реализован подсчет найденных велосипедов.
7. Добавлено отображение обработанного изображения с ограничивающими рамками.
8. Подключена база данных Supabase для хранения истории анализов.
9. Реализован просмотр истории выполненных обработок и экспорт результатов.
10. Исходный код проекта размещен в GitHub-репозитории.

При выполнении работы были рассмотрены существующие подходы к решению задач компьютерного зрения. К аналогичным решениям можно отнести системы видеонаблюдения с интеллектуальной аналитикой, облачные сервисы распознавания объектов и локальные приложения на основе предобученных моделей. Для проекта был выбран вариант самостоятельной разработки веб-приложения, так как он позволяет показать полный цикл работы AI-системы: от загрузки изображения до получения результата и сохранения данных.

Отчет состоит из введения, трех разделов, заключения, списка использованных источников и приложений. В первом разделе определяются объект, предмет и цель исследования, а также дается краткая характеристика объекта исследования. Во втором разделе описывается постановка практической задачи, используемые технологии, ход разработки веб-приложения и работа с базой данных. В третьем разделе представлены результаты выполнения индивидуального задания, описание работы готовой системы, тестирование и возможные направления улучшения проекта.

РАЗДЕЛ 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ ПРОЕКТА

1.1. Объект исследования

Объектом исследования является процесс учета велосипедов на парковке у станции метро. В рамках данной работы под учетом понимается определение количества велосипедов, находящихся на изображении парковочной зоны. Такой процесс может использоваться для оценки загруженности парковки и получения информации о том, насколько активно используется велосипедная инфраструктура около транспортного узла.

Велосипедная парковка у метро представляет собой участок городской инфраструктуры, предназначенный для временного размещения велосипедов. Пользователь оставляет велосипед на парковке и продолжает движение на общественном транспорте. Из-за этого такие парковки могут быть загружены неравномерно в разное время суток. Для анализа их состояния необходимо получать сведения о количестве велосипедов, находящихся на парковке.

В обычных условиях подсчет велосипедов может выполняться вручную. Такой способ не требует сложных технических средств, но имеет ограничения. При большом количестве объектов подсчет занимает больше времени. Если велосипеды частично перекрывают друг друга, находятся на заднем плане или изображение имеет недостаточное качество, вероятность ошибки увеличивается. Кроме того, ручной подсчет неудобен для регулярного контроля, так как каждый раз требует участия человека.

В рамках проекта рассматривалась задача обработки не видеопотока, а отдельного изображения. Пользователь выбирает фотографию парковки с компьютера, после чего система выполняет анализ. Такой формат был выбран потому, что он соответствует учебному заданию и позволяет реализовать полный цикл работы с изображением без необходимости подключать камеры или настраивать постоянное видеонаблюдение.

Для решения задачи был создан проект Velo-O-Wisp. Название проекта отражает его назначение в образной форме: часть «Velo» связана с велосипедной тематикой, а образ «блуждающего огонька» используется как

метафора системы, которая помогает выделить нужные объекты на изображении. Дизайн и название используются для оформления проекта и демонстрации, при этом основная часть работы связана с обработкой изображений, серверной логикой и сохранением результатов.

Объект исследования был выбран с учетом темы индивидуального задания. Учет велосипедов на парковке у метро является понятной прикладной задачей, которую можно решить с помощью компьютерного зрения. Для этого не требуется распознавать сложные действия человека или анализировать длительное видео. Достаточно обнаружить на изображении объекты нужного класса и подсчитать их количество.

Таким образом, объект исследования связан с автоматизацией визуального учета объектов городской инфраструктуры. В данном проекте этот объект был ограничен конкретной задачей — подсчетом велосипедов на фотографии парковки у метро.

1.2. Предмет исследования

Предметом исследования является веб-приложение компьютерного зрения, предназначенное для автоматического обнаружения и подсчета велосипедов на изображении. В отличие от объекта исследования, который связан с самой задачей учета велосипедов, предмет исследования описывает программное решение, с помощью которого эта задача выполняется.

Разработанная система включает несколько основных компонентов. Первый компонент — пользовательский интерфейс. Через него пользователь выбирает изображение с компьютера, запускает обработку и получает результат. Второй компонент — серверная часть. Она принимает изображение, передает его в модель искусственного интеллекта, сохраняет результат и возвращает данные на страницу. Третий компонент — нейронная сеть YOLOv8, которая выполняет детекцию объектов. Четвертый компонент — база данных Supabase, где сохраняется история выполненных анализов.

В рамках проекта предмет исследования связан не только с работой нейронной сети, но и с ее интеграцией в веб-приложение. Если использовать модель отдельно, результат можно получить в виде отдельного скрипта. Однако такой вариант неудобен для пользователя и не показывает работу полноценной информационной системы. Поэтому в проекте была реализована связка frontend, backend, модель обработки изображения и база данных.

Особое внимание было уделено тому, чтобы результат обработки был понятен пользователю. После анализа изображения система показывает количество найденных велосипедов и обработанное изображение, на котором найденные объекты выделены ограничивающими рамками. Такой способ отображения позволяет не только увидеть числовой результат, но и проверить, какие именно объекты были обнаружены моделью.

Также предмет исследования включает хранение результатов. Для этого в проекте используется таблица истории анализов. В ней сохраняются сведения о загруженном файле, дате обработки, количестве найденных велосипедов, времени обработки, исходном изображении и изображении с результатом. Это позволяет рассматривать проект не как разовый инструмент обработки фотографии, а как веб-систему, которая накапливает данные о выполненных анализах.

Таким образом, предмет исследования представляет собой программную реализацию системы учета велосипедов на основе компьютерного зрения. В проекте были объединены технологии веб-разработки, обработки изображений, нейронной детекции объектов и хранения данных.

1.3. Цель исследования

Целью исследования является разработка и проверка работы веб-приложения Velo-O-Wisp, которое выполняет автоматизированный учет велосипедов на парковке у метро по загруженному изображению.

Для достижения данной цели необходимо было создать систему, которая выполняет несколько последовательных действий. Сначала пользователь выбирает изображение с компьютера. Затем изображение передается на сервер. После этого сервер обрабатывает изображение с помощью модели YOLOv8, определяет объекты класса bicycle, подсчитывает их количество и формирует изображение с ограничивающими рамками. Далее результат отображается пользователю и сохраняется в базе данных.

В ходе работы цель исследования была связана с практической реализацией, а не только с изучением теоретических сведений о компьютерном зрении. Поэтому основное внимание было уделено разработке работающего приложения. Мной были настроены загрузка изображений, серверная обработка, интеграция модели YOLOv8, сохранение результатов и отображение истории анализов.

Для проекта были определены следующие функциональные требования:

1. Пользователь должен иметь возможность выбрать изображение парковки с компьютера.
2. Система должна отправлять изображение на серверную обработку.
3. Сервер должен выполнять детекцию велосипедов с использованием YOLOv8.
4. Система должна подсчитывать количество найденных объектов класса bicycle.
5. На обработанном изображении должны отображаться bounding boxes вокруг найденных велосипедов.
6. Результат обработки должен сохраняться в базе данных.
7. Пользователь должен иметь возможность просмотреть историю анализов.
8. Пользователь должен иметь возможность экспортировать результаты.

Кроме функциональных требований были учтены и общие требования к удобству работы. Интерфейс должен быть понятным для пользователя, результат обработки должен отображаться на странице, а запуск проекта должен быть возможен локально через сервер Flask. Также проект должен быть размещен в GitHub-репозитории, так как это требуется для сдачи исходного кода.

Цель исследования была достигнута за счет разработки веб-приложения, которое объединяет пользовательский интерфейс, серверную часть, модель YOLOv8, обработку изображений через OpenCV и базу данных Supabase. Полученная система позволяет загрузить изображение парковки, определить количество велосипедов, увидеть результат обработки и сохранить данные анализа.

РАЗДЕЛ 2. ПРАКТИЧЕСКАЯ РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

2.1. Постановка задач практической разработки

В практической части работы мной было разработано веб-приложение Velo-O-Wisp, предназначенное для автоматизированного учета велосипедов на парковке у метро. Основная задача приложения заключается в том, чтобы пользователь мог загрузить изображение парковки, запустить обработку и получить результат в виде количества найденных велосипедов и изображения с выделенными объектами.

При постановке задачи было определено, что система должна работать с изображениями, выбранными пользователем с компьютера. Такой вариант был выбран вместо обработки видеопотока, так как он соответствует учебному заданию и позволяет сосредоточиться на полном цикле обработки изображения. Пользователь не должен запускать Python-скрипты вручную или работать с командной строкой. Все основные действия должны выполняться через веб-интерфейс.

На первом этапе была определена логика работы приложения. Пользователь открывает сайт, выбирает фотографию парковки, нажимает кнопку запуска обработки, после чего изображение отправляется на сервер. Сервер принимает файл, передает его в нейронную сеть YOLOv8, получает результат распознавания, подсчитывает найденные велосипеды и формирует обработанное изображение с ограничивающими рамками. Затем результат возвращается пользователю и сохраняется в базе данных.

Для проекта были выделены следующие практические требования:

1. Реализовать загрузку изображения с компьютера пользователя.
2. Передавать выбранное изображение на серверную часть приложения.
3. Выполнять обработку изображения с использованием предобученной модели YOLOv8.
4. Находить на изображении объекты класса bicycle.
5. Подсчитывать количество найденных велосипедов.

6. Формировать изображение с bounding boxes вокруг найденных объектов.
7. Отображать пользователю исходный результат обработки в веб-интерфейсе.
8. Сохранять историю анализов в базе данных.
9. Реализовать просмотр истории выполненных обработок.
10. Добавить возможность экспорта истории результатов.
11. Подготовить проект к локальному запуску и размещению в GitHub-репозитории.

Также были определены ограничения проекта. Система работает с уже загруженным изображением, а не с онлайн-видеопотоком. Качество результата зависит от качества фотографии, освещения, ракурса, перекрытия велосипедов и возможностей предобученной модели. В рамках практики не выполнялось дополнительное обучение модели на специальном датасете велосипедных парковок, так как основной задачей было создание веб-приложения с интегрированной нейронной сетью.

При разработке было важно сделать проект не только работающим, но и удобным для демонстрации. Поэтому в приложении были предусмотрены главная страница, блок загрузки изображения, отображение результата, история анализов и экспорт данных. Такой набор функций позволяет показать не только саму работу модели, но и использование результата в рамках информационной системы.

2.2. Используемые технологии и архитектура проекта

Для реализации проекта были выбраны технологии, которые позволяют связать веб-интерфейс, серверную обработку изображения, нейронную сеть и базу данных. При выборе технологий учитывались требования практического задания, простота локального запуска и возможность показать работу системы в браузере.

Клиентская часть проекта была разработана с использованием HTML, CSS и JavaScript. HTML использовался для структуры страницы, CSS — для оформления интерфейса, JavaScript — для взаимодействия пользователя со страницей и отправки изображения на сервер. На стороне клиента пользователь выбирает файл, запускает обработку и получает результат без необходимости вручную выполнять команды в консоли.

Серверная часть была реализована на Python с использованием Flask. Flask был выбран потому, что он позволяет быстро создать веб-сервер, настроить маршруты и обработать HTTP-запросы. В проекте сервер принимает изображение, сохраняет его, передает в модель YOLOv8, получает результат обработки и возвращает данные на клиентскую часть.

Для детекции объектов использовалась библиотека Ultralytics YOLOv8. Модель YOLOv8 подходит для задачи поиска объектов на изображении. В проекте использовалась предобученная модель, которая уже содержит класс bicycle. Это позволило не обучать модель с нуля, а сосредоточиться на интеграции нейронной сети в веб-приложение.

Для работы с изображениями использовалась библиотека OpenCV. Она применялась для чтения изображений, обработки файлов и сохранения результата. После обработки изображение с найденными объектами сохраняется в папку результатов, а пользователь получает возможность увидеть его на странице сайта.

Для хранения истории анализов была подключена база данных Supabase. Supabase используется как облачная платформа с базой данных PostgreSQL. В проекте она применяется для сохранения сведений о каждом анализе: имени файла, дате загрузки, количестве найденных велосипедов, времени обработки, исходном изображении и изображении с результатом.

Общая архитектура проекта состоит из четырех основных частей:

1. Frontend — пользовательский интерфейс сайта.
2. Backend — серверная часть на Flask.
3. AI-модуль — обработка изображения с помощью YOLOv8.

4. База данных — хранение истории анализов в Supabase.

Работа системы строится по следующему сценарию. Пользователь открывает веб-страницу и выбирает изображение. JavaScript отправляет файл на сервер. Flask принимает изображение и передает его в модель YOLOv8. Модель выполняет детекцию объектов и возвращает информацию о найденных объектах. Затем сервер отбирает объекты класса bicycle, подсчитывает их количество и формирует изображение с ограничивающими рамками. После этого результат сохраняется и отображается пользователю.

Такая архитектура была выбрана потому, что она разделяет обязанности между частями проекта. Клиентская часть отвечает за внешний вид и действия пользователя. Серверная часть отвечает за обработку запросов и связь между компонентами. Нейронная сеть отвечает за обнаружение объектов. База данных отвечает за хранение результатов. Благодаря такому разделению проект проще проверять, дорабатывать и объяснять при защите.

2.3. Описание реализации проекта Velo-O-Wisp

Разработка проекта началась с подготовки структуры веб-приложения. Были созданы основные файлы проекта: серверный файл приложения, HTML-страница, CSS-файл оформления, файл зависимостей и папки для хранения загруженных и обработанных изображений. Такая структура позволяет отделить код приложения, статические файлы и результаты обработки.

Главная страница проекта была выполнена в единой визуальной стилистике. Название Velo-O-Wisp выбрано как образное название учебного проекта. Часть «Velo» указывает на велосипедную тематику, а образ «блуждающего огонька» используется как метафора системы, которая наблюдает за парковкой и подсвечивает найденные объекты на изображении. Дизайн используется для оформления и демонстрации проекта, а основная функциональность связана с обработкой изображений и сохранением результатов.

В интерфейсе была реализована возможность выбора изображения с компьютера. После выбора файла пользователь запускает обработку. На клиентской стороне формируется запрос к серверу, в который передается изображение. Это позволяет пользователю работать с системой через обычный браузер, не взаимодействуя напрямую с Python-кодом.

На серверной стороне был реализован прием изображения. Flask получает файл, сохраняет его и передает в обработку. После этого изображение передается в модель YOLOv8. Модель анализирует изображение и возвращает список найденных объектов. Среди найденных объектов выбираются велосипеды, после чего выполняется подсчет их количества.

Для визуализации результата используется изображение с bounding boxes. Ограничивающие рамки позволяют пользователю проверить, какие именно объекты были найдены системой. Это важно, потому что один только числовой результат не показывает, правильно ли модель определила велосипеды. Если на изображении есть сложные участки, перекрытия или похожие объекты, пользователь может визуально оценить результат обработки.

После завершения обработки сервер возвращает на страницу данные о результате. Пользователь видит количество найденных велосипедов и обработанное изображение. Также результат сохраняется в истории анализов. Благодаря этому можно не только выполнить один анализ, но и просмотреть список предыдущих обработок.

В проекте была реализована история выполненных анализов. Она позволяет сохранить информацию о каждом обработанном изображении. Это делает приложение более похожим на полноценную информационную систему, так как результат не теряется после закрытия страницы. Пользователь может вернуться к истории, посмотреть ранее обработанные изображения и сравнить результаты.

Также была добавлена возможность экспорта истории. Экспорт нужен для того, чтобы результаты можно было использовать вне приложения, например для отчета, проверки или дальнейшего анализа. В рамках учебного проекта эта функция демонстрирует, что система не только обрабатывает изображение, но и формирует данные, пригодные для последующего использования.

Проект был подготовлен к локальному запуску. Для установки зависимостей используется файл `requirements.txt`. Запуск сервера выполняется через Python-файл приложения или с помощью файла `run.bat`. После запуска пользователь может открыть сайт в браузере по локальному адресу и проверить работу системы.

2.4. Организация базы данных

Для сохранения истории анализов в проекте используется база данных Supabase. Она была подключена для того, чтобы результаты обработки не хранились только временно на странице, а записывались в отдельную таблицу. Такой подход позволяет фиксировать каждую выполненную обработку и использовать данные повторно.

В базе данных используется таблица `history`. Она предназначена для хранения сведений о выполненных анализах изображений. В таблицу записываются основные данные, которые нужны для отображения истории и проверки результата.

Структура таблицы `history` включает следующие поля:

1. `id` — уникальный идентификатор записи.
2. `file_name` — имя загруженного файла.
3. `upload_date` — дата и время загрузки изображения.
4. `bicycle_count` — количество найденных велосипедов.
5. `processing_time` — время обработки изображения.
6. `original_image` — ссылка или путь к исходному изображению.
7. `result_image` — ссылка или путь к обработанному изображению.

Поле `id` используется для различения записей между собой. Поле `file_name` позволяет понять, какое изображение было обработано. Поле `upload_date` фиксирует момент выполнения анализа. Поле `bicycle_count` содержит основной числовой результат работы системы. Поле `processing_time` нужно для оценки времени обработки изображения. Поля `original_image` и `result_image` используются для связи записи с исходным и обработанным изображением.

Сохранение истории анализов повышает практическую ценность проекта. Без базы данных приложение могло бы только обработать одно изображение и показать результат. С подключенной базой данных система сохраняет информацию о выполненных действиях и позволяет просматривать предыдущие результаты. Это соответствует задаче учета, так как учет предполагает не только разовое распознавание, но и фиксацию результата.

При разработке базы данных было важно выбрать простую и понятную структуру. Для учебного проекта не требовалось создавать большое количество связанных таблиц. Основная сущность в проекте — это запись анализа изображения. Поэтому одной таблицы `history` достаточно для хранения нужной информации.

В дальнейшем базу данных можно расширить. Например, можно добавить пользователя, станцию метро, адрес парковки, категорию изображения, статус проверки результата или комментариев. Также можно сохранять не только количество велосипедов, но и дополнительные параметры: среднюю уверенность модели, координаты найденных рамок и размер изображения. Однако для текущей версии проекта была выбрана минимальная структура, которая закрывает требования практического задания и не усложняет систему без необходимости.

Таким образом, во втором разделе была описана практическая часть разработки проекта `Velo-O-Wisp`. В ходе работы были определены требования к системе, выбраны технологии, реализованы клиентская и серверная части,

подключена модель YOLOv8, настроена обработка изображений и организовано хранение истории анализов в базе данных Supabase.

РАЗДЕЛ 3. РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

3.1. Описание результата практической деятельности

В результате выполнения индивидуального задания мной было разработано веб-приложение Velo-O-Wisp, предназначенное для автоматизированного учета велосипедов на парковке у метро. Проект выполняет полный цикл обработки изображения: пользователь загружает фотографию, сервер принимает файл, нейронная сеть YOLOv8 выполняет детекцию велосипедов, после чего система отображает обработанное изображение и сохраняет результат анализа в базе данных.

Готовое приложение позволяет пользователю выбрать изображение с компьютера и запустить обработку через веб-интерфейс. После запуска обработки файл передается на серверную часть, где выполняется анализ изображения. В качестве целевого класса используется bicycle, так как задача проекта связана именно с подсчетом велосипедов. После обработки пользователь получает количество найденных велосипедов и изображение с ограничивающими рамками вокруг обнаруженных объектов.

Основной результат работы проекта заключается в том, что задача учета велосипедов была переведена из ручного режима в автоматизированный. Пользователю не нужно самостоятельно подсчитывать объекты на фотографии. Достаточно загрузить изображение, после чего система выполняет обработку и возвращает результат. Это упрощает работу с изображениями парковок и позволяет использовать приложение как демонстрационный пример применения искусственного интеллекта в городской инфраструктуре.

В приложении также была реализована история выполненных анализов. После обработки изображения сведения о результате сохраняются в базе данных Supabase. В историю записываются имя файла, дата загрузки, количество найденных велосипедов, время обработки, исходное изображение

и обработанный результат. Благодаря этому можно не только выполнить один анализ, но и просмотреть ранее обработанные изображения.

Дополнительно была реализована возможность экспорта результатов. Эта функция нужна для того, чтобы данные истории можно было использовать вне приложения. Например, результаты можно сохранить и приложить к отчетным материалам или использовать для дальнейшего анализа.

Работа готового веб-приложения выполняется по следующему сценарию. Сначала пользователь открывает локальный сайт Velo-O-Wisp. Затем он выбирает изображение парковки у метро с компьютера. После нажатия кнопки запуска обработки изображение отправляется на сервер. Сервер выполняет анализ с помощью YOLOv8, сохраняет результат и возвращает пользователю информацию о количестве найденных велосипедов. После этого пользователь может перейти к истории и проверить сохраненную запись.

На рисунке 1 показана главная страница веб-приложения Velo-O-Wisp. На ней пользователь видит основное оформление проекта, название системы и элементы управления, необходимые для начала работы.

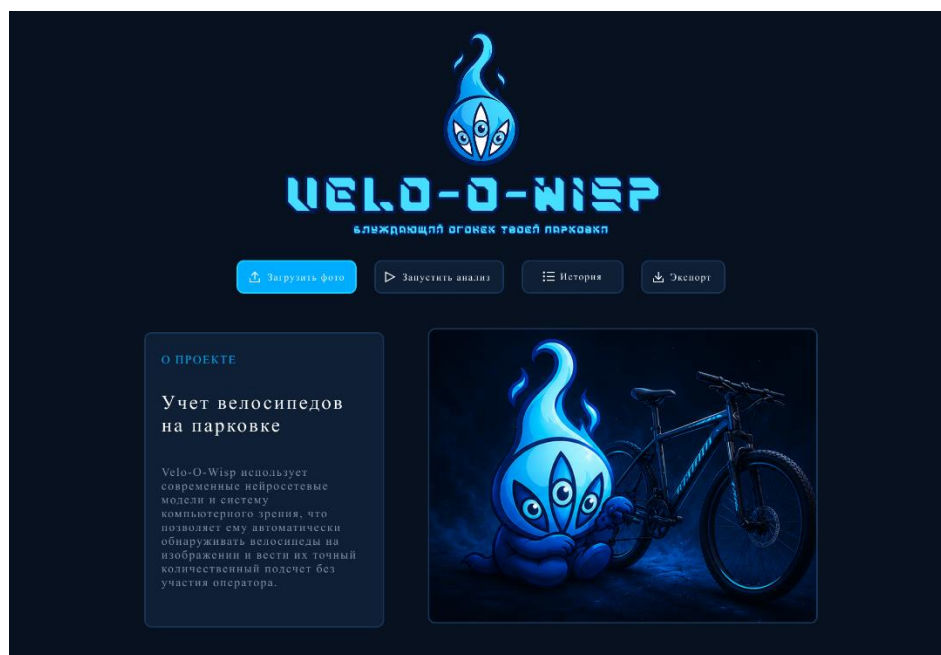


Рис. 1. Главная страница веб-приложения Velo-O-Wisp

На рисунке 2 представлен результат обработки изображения. Найденные велосипеды выделены ограничивающими рамками, а система отображает количество обнаруженных объектов.

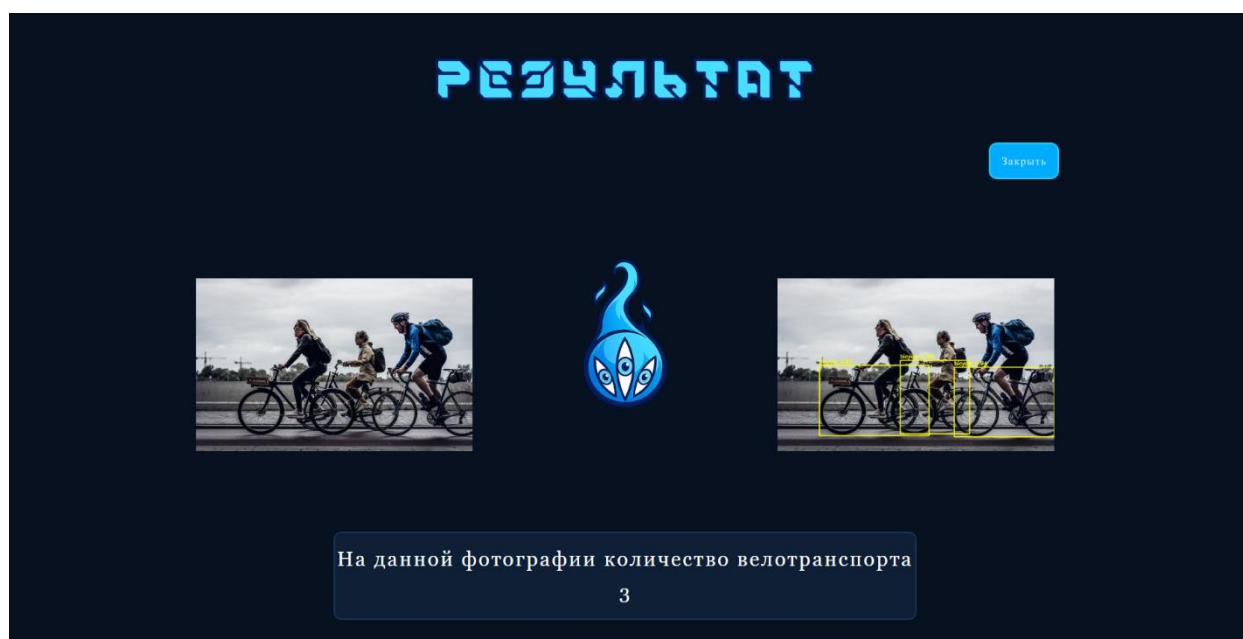


Рис. 2. Результат обработки изображения с найденными велосипедами

На рисунке 3 показана история выполненных анализов. В данном разделе пользователь может просмотреть ранее обработанные изображения и основные сведения о результатах.

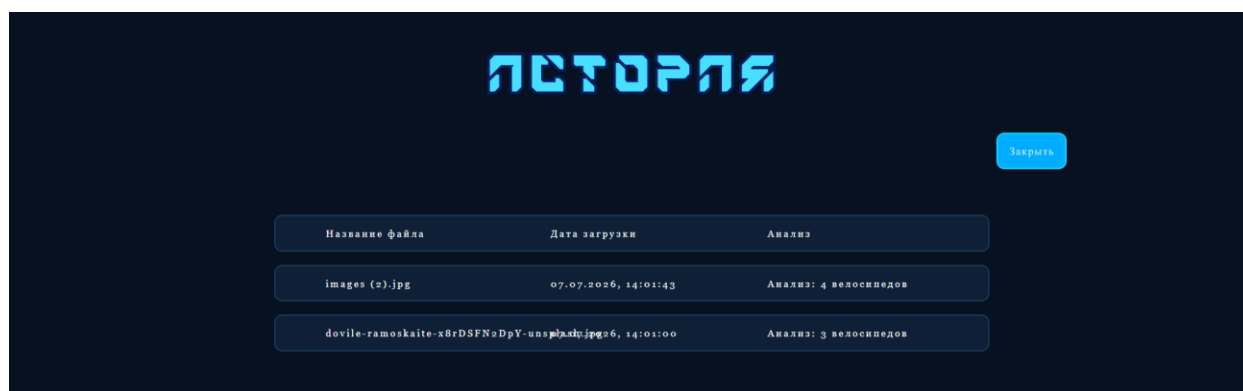


Рис. 3. История выполненных анализов

На рисунке 4 показана таблица history в базе данных Supabase. В ней сохраняются записи о выполненных анализах, включая имя файла, дату загрузки, количество найденных велосипедов и ссылки на изображения.

id	file_name	upload_date	bicycle_count	processing_time	result_image	original_image
17	dovile-ramoskate-x8rDSFN2DpI-uns	2026-07-07 14:01:00.64841	3	2.48	/static/results/result_967058ee2df64f2	/static/uploads/original_967058ee2df6
18	Images (2).jpg	2026-07-07 14:01:43.3808	4	0.09	/static/results/result_1a1895801ac4418f	/static/uploads/original_1a1895801ac44

Рис. 4. Таблица history в базе данных Supabase

Таким образом, в ходе практической части был получен работающий учебный проект, который соответствует теме индивидуального задания и демонстрирует применение нейронной сети для решения задачи детекции объектов.

3.2. Тестирование и анализ результатов

После завершения разработки было проведено тестирование веб-приложения. Цель тестирования заключалась в проверке основных функций системы: запуска приложения, загрузки изображения, обработки файла, подсчета велосипедов, отображения результата, сохранения истории и экспорта данных.

Проверка начиналась с локального запуска проекта. Для этого были установлены зависимости из файла requirements.txt, после чего сервер был запущен через Python-файл приложения. После запуска сайт открывался в браузере по локальному адресу. На этом этапе проверялось, что веб-интерфейс отображается корректно, стили подключены, кнопки доступны, а пользователь может выбрать изображение с компьютера.

Следующим этапом тестировалась загрузка изображения. В приложение были загружены изображения парковки, на которых присутствовали велосипеды. Система принимала файл и передавала его на серверную обработку. При корректной загрузке изображение обрабатывалось моделью YOLOv8, после чего на странице отображался результат.

Затем проверялась работа детекции. Модель должна была находить на изображении объекты класса bicycle и выделять их ограничивающими рамками. После обработки сравнивались исходное изображение и

изображение с результатом. Это позволило визуально проверить, какие объекты были обнаружены системой. Также проверялось, совпадает ли количество найденных объектов с количеством рамок на обработанном изображении.

Отдельно была проверена работа истории анализов. После каждой обработки результат должен был сохраняться в базе данных Supabase. В таблице history проверялось наличие новой записи, имя файла, дата загрузки, количество найденных велосипедов и ссылки на изображения. Это подтвердило, что приложение не только показывает результат на странице, но и сохраняет его для дальнейшего просмотра.

Также была проверена функция экспорта истории. Экспорт позволяет получить результаты анализов в отдельном файле. Эта функция была добавлена для удобства работы с накопленными данными и демонстрации того, что система может не только выполнять обработку изображения, но и формировать данные для дальнейшего использования.

В ходе тестирования были проверены следующие сценарии:

1. Запуск приложения на локальном сервере.
2. Открытие главной страницы в браузере.
3. Загрузка изображения с компьютера.
4. Обработка изображения с помощью YOLOv8.
5. Отображение количества найденных велосипедов.
6. Отображение изображения с bounding boxes.
7. Сохранение результата в базе данных Supabase.
8. Просмотр истории выполненных анализов.
9. Проверка записей в таблице history.
10. Экспорт истории результатов.

По результатам тестирования основные функции приложения работали корректно. Пользователь мог загрузить изображение, получить обработанный результат, увидеть количество найденных велосипедов и открыть историю

анализов. Также было подтверждено, что данные сохраняются в базе данных Supabase.

При анализе результатов было выявлено, что точность работы системы зависит от качества исходного изображения. Если велосипед хорошо виден, не перекрыт другими объектами и находится в нормальном ракурсе, модель определяет его корректно. Если велосипед частично закрыт, расположен далеко на изображении или сливается с фоном, результат может быть менее точным. Это является особенностью работы предобученных моделей компьютерного зрения и не связано только с реализацией веб-приложения.

Также было отмечено, что модель YOLOv8 может находить не все велосипеды на сложных изображениях. Например, если несколько велосипедов стоят очень близко друг к другу, часть объектов может быть определена как один объект или не обнаружена. Для повышения точности в будущем можно использовать дополнительное обучение модели на изображениях именно велосипедных парковок.

Несмотря на эти ограничения, разработанная система выполняет основную задачу проекта. Она позволяет автоматизировать первичный учет велосипедов на фотографии, визуально показывает результат распознавания и сохраняет историю анализов.

3.3. Ограничения проекта и возможные улучшения

Разработанный проект является учебным веб-приложением и имеет ряд ограничений. Первое ограничение связано с тем, что система работает только с изображениями, загруженными пользователем с компьютера. В текущей версии не реализована обработка видеопотока с камеры или автоматическая загрузка изображений из внешнего источника. Такой формат был выбран осознанно, так как в рамках практики требовалось реализовать понятный и проверяемый сценарий обработки изображения.

Второе ограничение связано с использованием предобученной модели. YOLOv8 уже содержит класс `bicycle`, поэтому модель может обнаруживать

велосипеды без дополнительного обучения. Однако она не была специально дообучена на фотографиях парковок у метро. Из-за этого в сложных условиях возможны ошибки: пропуск части велосипедов, неточное выделение рамок или неверное определение объекта.

Третье ограничение связано с качеством входных данных. Если изображение имеет низкое разрешение, сильное размытие, плохое освещение или большое количество перекрытий, качество распознавания может снижаться. Для более точной работы системы желательно использовать фотографии, на которых велосипеды видны достаточно четко.

Четвертое ограничение связано с тем, что система считает только найденные объекты класса `bicycle`. Она не анализирует свободные места, тип парковки, плотность размещения велосипедов и изменение загруженности во времени. Для текущей версии проекта этого достаточно, так как индивидуальное задание связано именно с учетом велосипедов на изображении.

В дальнейшем проект можно улучшить несколькими способами. Во-первых, можно добавить обработку видеопотока. Это позволило бы использовать систему не только для разовой фотографии, но и для регулярного мониторинга парковки. Во-вторых, можно дообучить модель на отдельном наборе изображений велосипедных парковок. Это повысило бы точность обнаружения в условиях, характерных именно для данной задачи.

Также можно расширить базу данных. Например, добавить информацию о станции метро, адресе парковки, комментарии к анализу, статусе проверки результата и координатах найденных объектов. Это позволило бы использовать историю не только как список обработанных изображений, но и как основу для статистического анализа.

Еще одним возможным улучшением является добавление авторизации пользователей. В текущей версии проект работает как учебное приложение, но при дальнейшем развитии можно сделать отдельные учетные записи,

чтобы каждый пользователь видел только свои результаты анализа. Также можно добавить роли администратора и обычного пользователя.

Дополнительно можно улучшить экспорт результатов. Например, сформировать отчет в формате Excel или PDF, где будут отображаться дата анализа, имя файла, количество найденных велосипедов и ссылка на обработанное изображение. Это сделало бы систему удобнее для использования в отчетности.

В качестве направления развития можно добавить сравнение результатов за разные даты. Если парковка анализируется регулярно, система сможет показывать, в какие дни или часы она наиболее загружена. Такой функционал уже выходит за рамки учебного задания, но логически продолжает идею автоматизированного учета.

Таким образом, проект Velo-O-Wisp был выполнен как рабочее веб-приложение для детекции и подсчета велосипедов на изображении. В ходе работы были реализованы основные функции, необходимые для выполнения индивидуального задания: загрузка изображения, обработка с помощью YOLOv8, отображение bounding boxes, подсчет найденных объектов, сохранение истории, проверка записей в базе данных и экспорт результатов. Несмотря на ограничения, система демонстрирует практическое применение технологий искусственного интеллекта и может быть расширена в дальнейшем.

ЗАКЛЮЧЕНИЕ

В ходе производственной практики мной был разработан учебный веб-проект Velo-O-Wisp по теме «Учет велосипедов на парковке у метро». Цель работы была достигнута: создано веб-приложение, которое позволяет загрузить изображение, выполнить детекцию велосипедов с помощью YOLOv8, вывести результат обработки и сохранить данные анализа в базе данных Supabase.

В процессе работы были реализованы клиентская часть сайта, серверная часть на Flask, обработка изображений через YOLOv8 и OpenCV, подсчет найденных объектов класса bicycle, отображение bounding boxes, история анализов и экспорт результатов. Исходный код проекта был размещен в GitHub-репозитории.

По результатам тестирования приложение корректно выполняет основные функции: принимает изображение, обрабатывает его, отображает количество найденных велосипедов, сохраняет результат в базе данных и позволяет просматривать историю обработок.

К ограничениям проекта можно отнести зависимость качества распознавания от исходного изображения, освещения, ракурса и степени перекрытия велосипедов. В дальнейшем проект можно улучшить за счет дообучения модели на изображениях велосипедных парковок, добавления обработки видеопотока, расширения базы данных и формирования отчетов в PDF или Excel.

Таким образом, в ходе практики мной был выполнен полный цикл разработки учебной AI-системы для обработки изображений: от постановки задачи и выбора модели до создания веб-приложения, тестирования и подготовки проекта к сдаче.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ultralytics. YOLOv8 Documentation — <https://docs.ultralytics.com/models/yolov8/>
2. Ultralytics. Ultralytics Documentation — <https://docs.ultralytics.com/>
3. Pallets Projects. Flask Documentation — <https://flask.palletsprojects.com/>
4. Pallets Projects. Flask Quickstart — <https://flask.palletsprojects.com/en/stable/quickstart/>
5. OpenCV. OpenCV Documentation — <https://docs.opencv.org/4.x/>
6. Python Software Foundation. Python Documentation — <https://docs.python.org/3/>
7. Supabase. Database Documentation — <https://supabase.com/docs/guides/database/overview>
8. PostgreSQL Global Development Group. PostgreSQL Documentation — <https://www.postgresql.org/docs/>
9. MDN Web Docs. HTML: HyperText Markup Language — <https://developer.mozilla.org/en-US/docs/Web/HTML>
10. MDN Web Docs. Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

ПРИЛОЖЕНИЯ

Приложение А. Репозиторий проекта

Исходный код веб-приложения Velo-O-Wisp размещен в GitHub-репозитории:

<https://github.com/Apolion/Velo-O-Wisp/tree/main>

В репозитории представлены основные файлы проекта: серверная часть приложения, клиентская часть, файлы стилей, зависимости проекта, скрипт запуска и материалы, необходимые для локальной проверки работы веб-приложения.

Приложение Б. Макет интерфейса проекта

Макет пользовательского интерфейса веб-приложения Velo-O-Wisp размещен в Figma:

<https://www.figma.com/design/BUpYBlQVRnp3zMPwtZ1cGn/Untitled?node-id=7-138&t=AI9JqU0fAS2sRs96-0>

Макет использовался как основа для оформления главной страницы, страницы результата обработки и раздела истории анализов.